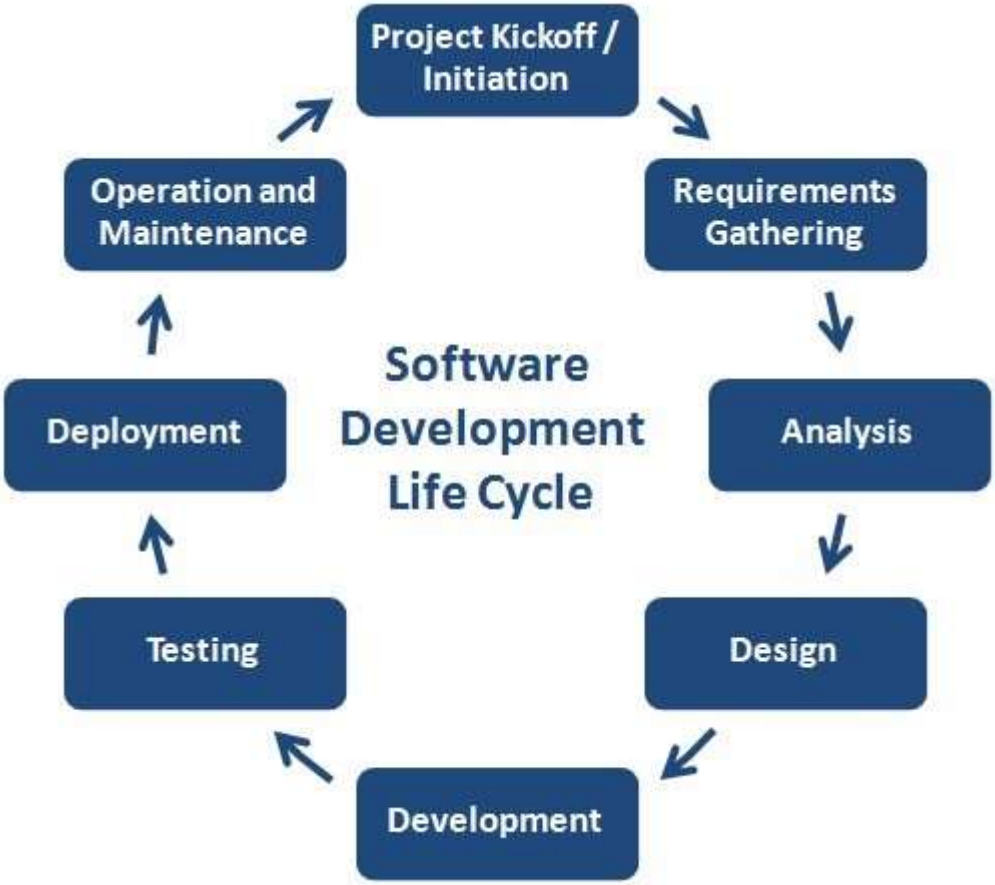
SDLC

The Software Development Life Cycle (SDLC) is a structured framework used by software organizations to design, develop, and test high-quality software. It defines the entire process of software production, from the initial idea to the final deployment and maintenance.



1. **Planning:** Defines project scope, goals, resources, costs, and timeline. It also involves feasibility studies (technical, operational, economic).
2. **Requirements Analysis:** Gathers detailed business requirements from stakeholders, often producing a Software Requirement Specification (SRS) document.
3. **Design:** Translates requirements into a technical blueprint, covering system architecture, user interface (UI), database structure, and technology stack.
4. **Coding (Development):** Developers write the actual source code based on the design specifications.
5. **Testing:** QA engineers identify bugs and verify that the software meets the initial requirements through unit, integration, and user acceptance testing.
6. **Deployment:** The software is released to the production environment, making it available to users.
7. **Maintenance:** Ongoing support, bug fixing, and updates to ensure the software continues to function optimally and meets changing needs.

Assertion

Debugging Python Program by using assert Keyword:

- The process of identifying and fixing the bug is called debugging.

- Very common way of debugging is to use print () statement. But the problem with the print () statement is after fixing the bug, compulsory we have to delete the extra added print () statements, otherwise these will be executed at runtime which creates performance problems and disturbs console output.

- To overcome this problem, we should go for assert statement. The main advantage of assert statement over print () statement is after fixing bug we are not required to delete assert statements. Based on our requirement we can enable or disable assert statements.

- Hence the main purpose of assertions is to perform debugging. Usually we can perform debugging either in development or in test environments but not in production environment. Hence assertions concept is applicable only for dev and test environments but not for production environment.

```
# initializing number
a = 4
b = 0

# using assert to check for 0
print("The value of a / b is : ")
assert b != 0
print(a / b)
```

```
# Python 3 code to demonstrate
# working of assert

# initializing number
a = 4
b = 0

# using assert to check for 0
print("The value of a / b is : ")
assert b != 0, "Zero Division Error"
print(a / b)
```

Syntax

The basic syntax is simple and has two forms: Real Python

- `assert condition` : If the `condition` is `False` , raises an `AssertionError` with no message.
- `assert condition, message` : If the `condition` is `False` , raises an `AssertionError` with the custom `message` provided.

Ques

Reverse a Number

```
num = 1234

reversed_num = 0


while num != 0:

    digit = num % 10

    reversed_num = reversed_num * 10 + digit

    num //= 10


print("Reversed Number: " + str(reversed_num))
```

Prime number

```
num = 29

# To take input from the user
#num = int(input("Enter a number: "))

# define a flag variable
flag = False

if num == 0 or num == 1:
    print(num, "is not a prime number")
elif num > 1:
    # check for factors
    for i in range(2, num):
        if (num % i) == 0:
            # if factor is found, set flag to True
            flag = True
            # break out of loop
            break

    # check if flag is True
    if flag:
        print(num, "is not a prime number")
    else:
        print(num, "is a prime number")
```

For Practice:

https://www.hackerrank.com/domains/python

Reverse a sentence:

S = “My name is Python”

```
S = "My name is Python"
# 1. Split
words = S.split()
# 2. Reverse
words.reverse()
# 3. Join
result = " ".join(words)
print(result) # Output: "Python is name My"
```

Reverse each character

```
S = "My name is Python"
reversed_S = ""
i = len(S) - 1

while i >= 0:
    # Logic: Append character from the end
    reversed_S += S[i]
    i -= 1

print(reversed_S)
# Output: "nohtyP si eman yM"
```